

Assignment-19.5

Name:G.Charanya

HT NO:2303A510G6

Batch-23

Task 1 – Python List Processing → C++ STL Conversion

Task:

Translate a Python program that filters even numbers from a list into C++.

Instructions:

- Provide AI with Python list comprehension code.
- Convert it into C++ using vectors and loops/STL.
- Verify outputs for the same input list.

Expected Output:

- Equivalent working C++ program using STL.

```
1  #generate a python code that filters even numbers from a list
2  from ast import List
3  def filter_even_numbers(numbers: List[int]) -> List[int]:
4      """ Filters even numbers from a list of integers.
5      Args:
6          numbers (List[int]): A list of integers.
7      Returns:
8          List[int]: A list of even integers.
9      """
10     even_numbers = [num for num in numbers if num % 2 == 0]
11     return even_numbers
12 # Example usage
13 if __name__ == "__main__":
14     sample_numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
15     even_numbers = filter_even_numbers(sample_numbers)
16     print(f"Even numbers in the list: {even_numbers}")
17 #convert the above code to c++ using vectors and loops/STL.
18 #include <iostream>
19 #include <vector>
20 using namespace std;
21 vector<int> filterEvenNumbers(const vector<int>& numbers) {
```

```

20 using namespace std;
21 vector<int> filterEvenNumbers(const vector<int>& numbers) {
22     vector<int> evenNumbers;
23     for (int num : numbers) {
24         if (num % 2 == 0) {
25             evenNumbers.push_back(num);
26         }
27     }
28     return evenNumbers;
29 }
30 int main() {
31     vector<int> sampleNumbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
32     vector<int> evenNumbers = filterEvenNumbers(sampleNumbers);
33     cout << "Even numbers in the list: ";
34     for (int num : evenNumbers) {
35         cout << num << " ";
36     }
37     cout << endl;
38     return 0;
39 }

```

Task 2 – Java to Python OOP Translation

Task:

Convert a Java Student class into an equivalent Python class.

Instructions:

- Translate attributes, constructor, and methods properly.
- Ensure Python version uses `__init__` and follows Python conventions.
- Test both versions with sample objects.

Expected Output:

- Correct Python OOP class translated from Java

```

1  #generate a java code with class and respective attributes
2  public class Student {
3      private String name;
4      private int age;
5      private String email;
6      public Student(String name, int age, String email) {
7          this.name = name;
8          this.age = age;
9          this.email = email;
10     }
11     // Getters and setters
12     public String getName() {
13         return name;
14     }
15     public void setName(String name) {
16         this.name = name;
17     }
18     public int getAge() {
19         return age;
20     }
21     public void setAge(int age) {
22         this.age = age;
23     }
24     public String getEmail() {
25         return email;
26     }
27     public void setEmail(String email) {
28         this.email = email;
29     }
30 }
31 #convert the above java code to python code

```

```

29
30 }
31 #convert the above java code to python code
32 class Student:
33     def __init__(self, name, age, email):
34         self.name = name
35         self.age = age
36         self.email = email
37     # Getters and setters
38     def get_name(self):
39         return self.name
40     def set_name(self, name):
41         self.name = name
42     def get_age(self):
43         return self.age
44     def set_age(self, age):
45         self.age = age
46     def get_email(self):
47         return self.email
48     def set_email(self, email):
49         self.email = email
50 #example usage
51 student1 = Student("Alice", 20, "alice@example.com")
52 print(student1.get_name()) # Output: Alice
53 print(student1.get_age()) # Output: 20
54 print(student1.get_email()) # Output: alice@example.com
55

```

Task 3 – SQL Aggregation Query → Pandas GroupBy

Task:

Convert an SQL query with aggregation into Pandas.

Example SQL:

```
SELECT department, AVG(salary)
```

```
FROM employees
```

```
GROUP BY department;
```

Instructions:

- Translate into Pandas `groupby()` and `mean()`.
- Validate using sample DataFrame.

Expected Output:

- Equivalent Pandas aggregation code.

```
1  #create a SQL query to find the average salary of employees in each department from a table 'employees'. Group the results by department
2  SELECT department, AVG(salary) AS average_salary
3  FROM employees
4  GROUP BY department;
5  #convert the above code to python code using pandas library
6  import pandas as pd
7  # Assuming you have a DataFrame 'df' with columns 'department' and 'salary'
8  average_salary_by_department = df.groupby('department')['salary'].mean().reset_index()
9  average_salary_by_department.rename(columns={'salary': 'average_salary'}, inplace=True)
10 print(average_salary_by_department)
```

Task 4 – Python File Handling → C++ File Streams

Task:

Convert a Python program that reads a text file and counts words into C++.

Instructions:

- Translate `open()` usage into `ifstream`.
- Handle file-not-found errors properly.
- Compare outputs for the same file.

Expected Output:

- Working C++ word counter program.

```

1  #generate a pythn code that reads a text file and counts words
2  def count_words_in_file(file_path):
3      try:
4          with open(file_path, 'r') as file:
5              text = file.read()
6              words = text.split()
7              word_count = len(words)
8              return word_count
9      except FileNotFoundError:
10         return "File not found. Please check the file path."
11  # Example usage
12  file_path = input("Enter the path to the text file: ")
13  result = count_words_in_file(file_path)
14  print(f"The number of words in the file is: {result}")
15  #convert the above code into c++
16  #include <iostream>
17  #include <fstream>
18  #include <sstream>
19  #include <string>
20  using namespace std;
21  int countWordsInFile(const string& filePath) {
22      ifstream file(filePath);
23      if (!file.is_open()) {

```

```

20  using namespace std;
21  int countWordsInFile(const string& filePath) {
22      ifstream file(filePath);
23      if (!file.is_open()) {
24          cerr << "File not found. Please check the file path." << endl;
25          return -1;
26      }
27      string word;
28      int wordCount = 0;
29      while (file >> word) {
30          wordCount++;
31      }
32      file.close();
33      return wordCount;
34  }
35  int main() {
36      string filePath;
37      cout << "Enter the path to the text file: ";
38      getline(cin, filePath);
39      int result = countWordsInFile(filePath);
40      if (result != -1) {
41          cout << "The number of words in the file is: " << result << endl;
42      }
43      return 0;
44  }
45

```

Task 5 – Java Exception Handling → Python Try/Except

Task:

Translate a Java program using try-catch for division into Python.

Instructions:

- Convert exception syntax properly.
- Handle divide-by-zero gracefully.
- Verify both programs behave the same.

Expected Output:

- Equivalent Python error-handling program.

```

1  #generate a java code using try-catch for division
2  import java.util.Scanner;
3  public class Division {
4      public static void main(String[] args) {
5          Scanner scanner = new Scanner(System.in);
6          System.out.print("Enter the first number: ");
7          double num1 = scanner.nextDouble();
8          System.out.print("Enter the second number: ");
9          double num2 = scanner.nextDouble();
10         try {
11             double result = num1 / num2;
12             System.out.println("The result of the division is: " + result);
13         } catch (ArithmeticException e) {
14             System.out.println("Error: Division by zero is not allowed.");
15         }
16     }
17 }
18 #convert above code into python code using try-except for division
19 num1=float(input("Enter the first number: "))
20 num2=float(input("Enter the second number: "))
21 try:
22     result=num1/num2
23     print(f"The result of the division is: {result}")
24 except ZeroDivisionError:
25     print("Error: Division by zero is not allowed.")

```